



UFRJ



INSTITUTO DE
COMPUTAÇÃO
UFRJ

Programação de Computadores II

Métodos de Ordenação

Prof. Marcos Tomazzoli Leipnitz

Sumário

- Algoritmos de Ordenação por Divisão e Conquista
 - *Merge Sort*
- Complexidade de Algoritmos

Tipos de algoritmos de ordenação

- Ordenação por comparação e troca:
 - **Bubble Sort** – método da bolha - os elementos vão sendo ordenados do maior para o menor
 - **Insertion Sort** – os elementos são ordenados a partir do início do vetor
 - **Selection Sort** – passando pelo vetor, seleciona o menor (ou o maior) valor para a primeira posição, depois o segundo menor (ou maior), até os dois últimos elementos
- Ordenação por Divisão e Conquista:
 - **Merge Sort** – divide o vetor em partes (subproblemas), resolvendo esses vários subproblemas e depois une as soluções dos subproblemas

Merge Sort - Algoritmo

- **Base:** $n = 1$. Um conjunto de um único elemento está ordenado
- **H.I.:** Sei ordenar um conjunto de $1 \leq k < n$ valores
- **Passo:**
 - Seja S um conjunto de $n \geq 2$ valores. Podemos particionar S em dois conjuntos, $S1$ e $S2$, de tamanhos $n/2$
 - Pela **H.I.**, sabemos ordenar os conjuntos $S1$ e $S2$. Podemos então obter S ordenado intercalando os conjuntos ordenados $S1$ e $S2$

Merge Sort

- Dividir:
 - Divida o arranjo de n elementos em dois sub-arranjos de $n/2$ elementos cada
- Conquistar:
 - Ordene os dois sub-arranjos recursivamente
- Combinar:
 - Intercale os dois sub-arranjos ordenados, de modo a produzir a resposta ordenada

Merge Sort

- Podemos ver que a divisão é imediata
 - O arranjo é dividido em dois sub-arranjos com metade do tamanho do original, que são ordenados recursivamente
 - A conquista também não é difícil
 - Inevitavelmente vamos parar no caso base, que é trivial
 - E a combinação?
 - Como intercalar os 2 sub-arranjos ordenados?
-

Merge - Intercalação

```
void merge(int A[], int p, int q,
int r) {
    int i, j, k;
    int tamseq1 = q - p + 1;
    int tamseq2 = r - q;

    int seq1[tamseq1];
    for(i=0; i < tamseq1; i++)
        seq1[i] = A[p+i];
    int seq2[tamseq2];
    for(j=0; j < tamseq2; j++)
        seq2[j] = A[q+1+j];
    k = p; i = 0; j = 0;
    while (i < tamseq1 && j < tamseq2)
    {
        if(seq2[j] <= seq1[i]) {
            A[k] = seq2[j];
            j++; }
        else {
            A[k] = seq1[i];
            i++; }
        k++;
    }
    while (i < tamseq1) {
        A[k] = seq1[i];
        k++;
        i++;
    }
    while (j < tamseq2) {
        A[k] = seq2[j];
        k++;
        j++;
    }
}
```

Merge - Exemplo

- Considere o arranjo **A** e os índices **p**, **q** e **r**:

A	3	12	1	8	4	6	10	5	7	12	15	5	2	1	8
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
					↑ p		↑ q				↑ r				

Merge - Exemplo

- Considere o arranjo **A** e os índices **p**, **q** e **r**:

A	3	12	1	8	4	6	10	5	7	12	15	5	2	1	8
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
					↑ p		↑ q				↑ r				

Note que **A[p . . q]** e **A[q+1 . . r]** estão ordenados

Merge - Exemplo

- Considere o arranjo **A** e os índices **p**, **q** e **r**:

A	3	12	1	8	4	6	10	5	7	12	15	5	2	1	8
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
					↑ p		↑ q				↑ r				

seq1	4	6	10
------	---	---	----

seq2	5	7	12	15
------	---	---	----	----

Copiamos **A**[**p**..**q**] em **seq1** e **A**[**q**+1..**r**] em **seq2**

Merge - Exemplo

- Considere o arranjo **A** e os índices **p**, **q** e **r**:

A	3	12	1	8	4	6	10	5	7	12	15	5	2	1	8
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
					↑ p		↑ q				↑ r				

seq1	4	6	10
------	---	---	----

seq2	5	7	12	15
------	---	---	----	----

A[k] recebe o menor elemento dentre os 2 sub-arranjos

Merge - Exemplo

- Considere o arranjo **A** e os índices **p**, **q** e **r**:

A	3	12	1	8	4	6	10	5	7	12	15	5	2	1	8
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
					↑ p		↑ q				↑ r				

seq1	4	6	10
------	---	---	----

seq2	5	7	12	15
------	---	---	----	----

A[k] recebe o menor elemento dentre os 2 sub-arranjos

Merge - Exemplo

- Considere o arranjo **A** e os índices **p**, **q** e **r**:

A	3	12	1	8	4	6	10	5	7	12	15	5	2	1	8
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
					↑ p		↑ q				↑ r				

seq1	4	6	10
------	---	---	----

seq2	5	7	12	15
------	---	---	----	----

A[k] recebe o menor elemento dentre os 2 sub-arranjos

Merge - Exemplo

- Considere o arranjo **A** e os índices **p**, **q** e **r**:

A	3	12	1	8	4	6	10	5	7	12	15	5	2	1	8
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
					↑ p		↑ q				↑ r				

seq1	4	6	10
------	---	---	----

seq2	5	7	12	15
------	---	---	----	----

A[k] recebe o menor elemento dentre os 2 sub-arranjos

Merge - Exemplo

- Considere o arranjo **A** e os índices **p**, **q** e **r**:

A	3	12	1	8	4	5	10	5	7	12	15	5	2	1	8
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
					↑ p		↑ q				↑ r				

seq1	4	6	10
------	---	---	----

seq2	5	7	12	15
------	---	---	----	----

A[k] recebe o menor elemento dentre os 2 sub-arranjos

Merge - Exemplo

- Considere o arranjo **A** e os índices **p**, **q** e **r**:

A	3	12	1	8	4	5	10	5	7	12	15	5	2	1	8
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
					↑ p		↑ q				↑ r				

seq1	4	6	10
------	---	---	----

seq2	5	7	12	15
------	---	---	----	----

A[k] recebe o menor elemento dentre os 2 sub-arranjos

Merge - Exemplo

- Considere o arranjo **A** e os índices **p**, **q** e **r**:

A	3	12	1	8	4	5	6	5	7	12	15	5	2	1	8
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
					↑ p		↑ q				↑ r				

seq1	4	6	10
------	---	---	----

seq2	5	7	12	15
------	---	---	----	----

A[k] recebe o menor elemento dentre os 2 sub-arranjos

Merge - Exemplo

- Considere o arranjo **A** e os índices **p**, **q** e **r**:

A	3	12	1	8	4	5	6	5	7	12	15	5	2	1	8
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
					↑ p		↑ q				↑ r				

seq1	4	6	10
------	---	---	----

seq2	5	7	12	15
------	---	---	----	----

A[k] recebe o menor elemento dentre os 2 sub-arranjos

Merge - Exemplo

- Considere o arranjo **A** e os índices **p**, **q** e **r**:

A	3	12	1	8	4	5	6	7	7	12	15	5	2	1	8
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
					↑ p		↑ q				↑ r				

seq1	4	6	10
------	---	---	----

seq2	5	7	12	15
------	---	---	----	----

A[k] recebe o menor elemento dentre os 2 sub-arranjos

Merge - Exemplo

- Considere o arranjo **A** e os índices **p**, **q** e **r**:

A	3	12	1	8	4	5	6	7	7	12	15	5	2	1	8
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
					↑ p		↑ q				↑ r				

seq1	4	6	10
------	---	---	----

seq2	5	7	12	15
------	---	---	----	----

A[k] recebe o menor elemento dentre os 2 sub-arranjos

Merge - Exemplo

- Considere o arranjo **A** e os índices **p**, **q** e **r**:

A	3	12	1	8	4	5	6	7	10	12	15	5	2	1	8
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
					↑ p		↑ q				↑ r				

seq1	4	6	10
------	---	---	----

seq2	5	7	12	15
------	---	---	----	----

A[k] recebe o menor elemento dentre os 2 sub-arranjos

Merge - Exemplo

- Considere o arranjo **A** e os índices **p**, **q** e **r**:

A	3	12	1	8	4	5	6	7	10	12	15	5	2	1	8
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
					↑ p		↑ q				↑ r				

seq1	4	6	10
------	---	---	----

seq2	5	7	12	15
------	---	---	----	----

A[k] recebe o menor elemento dentre os 2 sub-arranjos

Merge - Exemplo

- Considere o arranjo **A** e os índices **p**, **q** e **r**:

A	3	12	1	8	4	5	6	7	10	12	15	5	2	1	8
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
					↑ p		↑ q				↑ r				

seq1	4	6	10
------	---	---	----

seq2	5	7	12	15
------	---	---	----	----

A[k] recebe o menor elemento dentre os 2 sub-arranjos

Merge - Exemplo

- Considere o arranjo **A** e os índices **p**, **q** e **r**:

A	3	12	1	8	4	5	6	7	10	12	15	5	2	1	8
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
					↑ p		↑ q				↑ r				

seq1	4	6	10
------	---	---	----

seq2	5	7	12	15
------	---	---	----	----

A[k] recebe o menor elemento dentre os 2 sub-arranjos

Merge - Exemplo

- Considere o arranjo **A** e os índices **p**, **q** e **r**:

A	3	12	1	8	4	5	6	7	10	12	15	5	2	1	8
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
					↑ p		↑ q				↑ r				

seq1	4	6	10
------	---	---	----

seq2	5	7	12	15
------	---	---	----	----

A[k] recebe o menor elemento dentre os 2 sub-arranjos

Merge - Exemplo

- Considere o arranjo **A** e os índices **p**, **q** e **r**:

A	3	12	1	8	4	5	6	7	10	12	15	5	2	1	8
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
					↑ p		↑ q				↑ r				

seq1	4	6	10
------	---	---	----

seq2	5	7	12	15
------	---	---	----	----

E **A**[**p** . . **r**] está ordenado

Merge Sort - Completo

```
void mergeSort(int numeros[], int ini,
int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini, meio);
        mergeSort(numeros, meio+1, fim);

        merge(numeros, ini, meio, fim);
    }
}
```

Merge Sort - Completo

```
void mergeSort(int numeros[], int ini,
int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;
        mergeSort(numeros, ini, meio);
        mergeSort(numeros, meio+1, fim);
        merge(numeros, ini, meio, fim);
    }
}
```

Dividir

Conquistar

Combinar

Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

2	8	7	1	3	5	6	4
0	1	2	3	4	5	6	7
↑							↑
ini							fim

Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

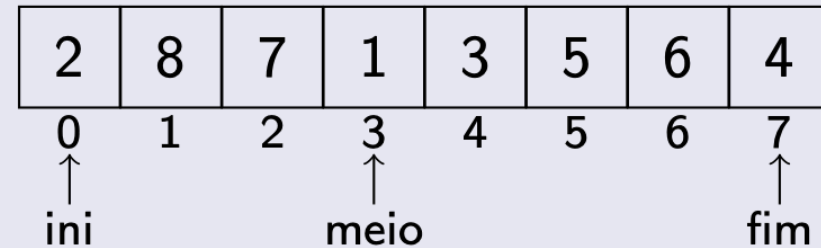
2	8	7	1	3	5	6	4
0	1	2	3	4	5	6	7
↑							↑
ini							fim

Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

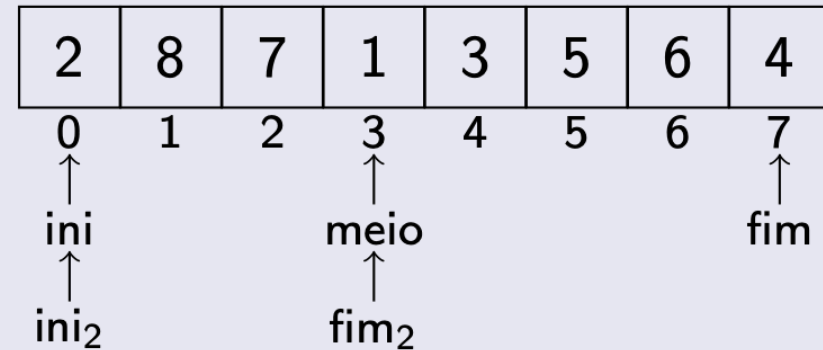


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

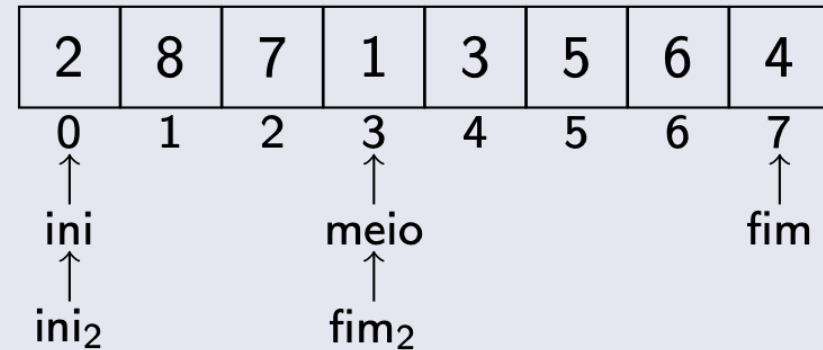


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

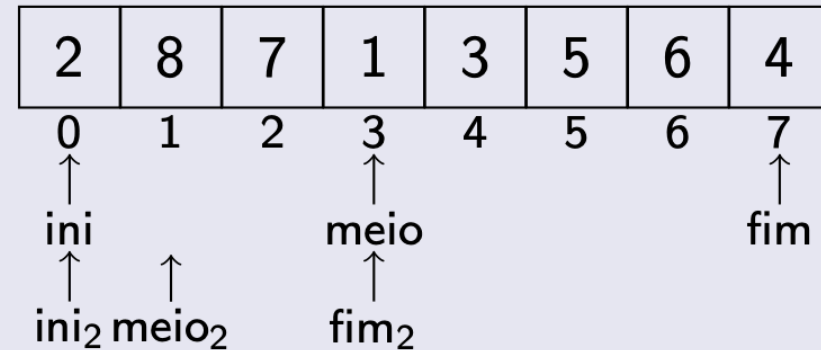


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

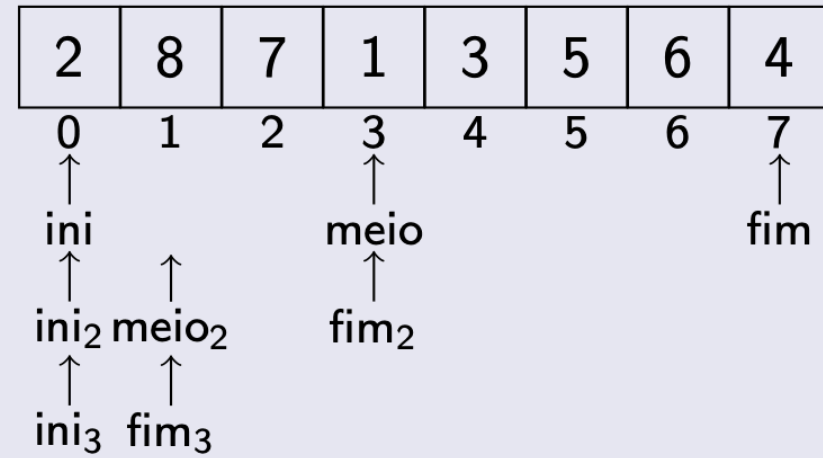


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

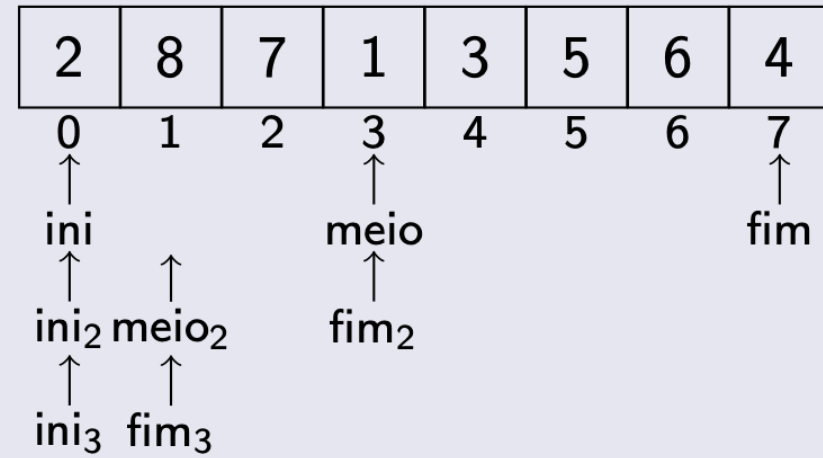


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

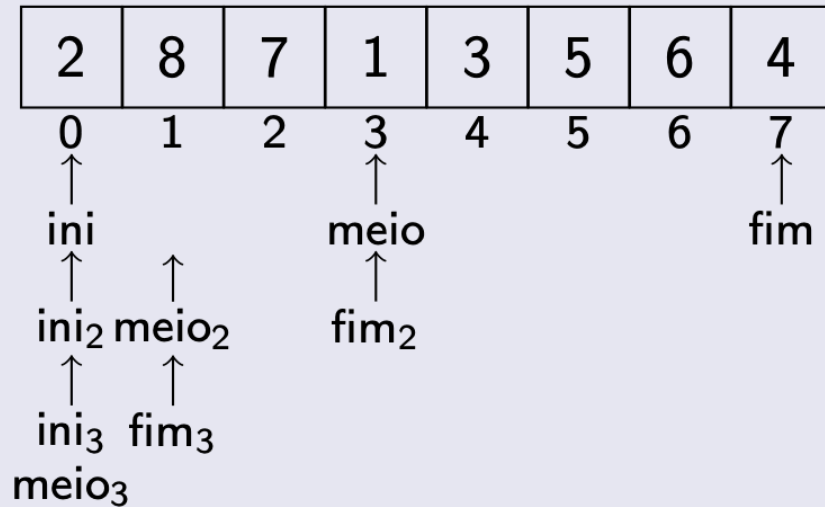


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

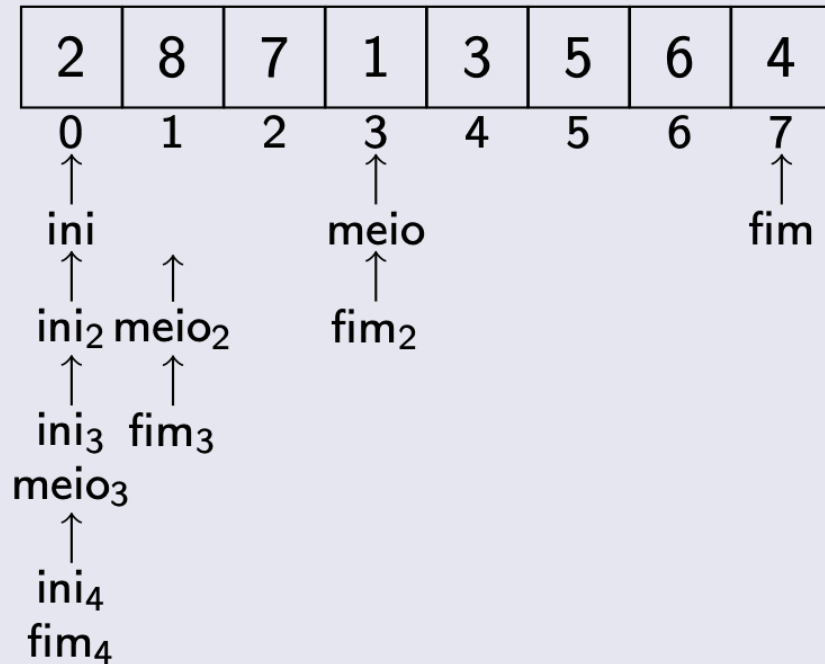


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

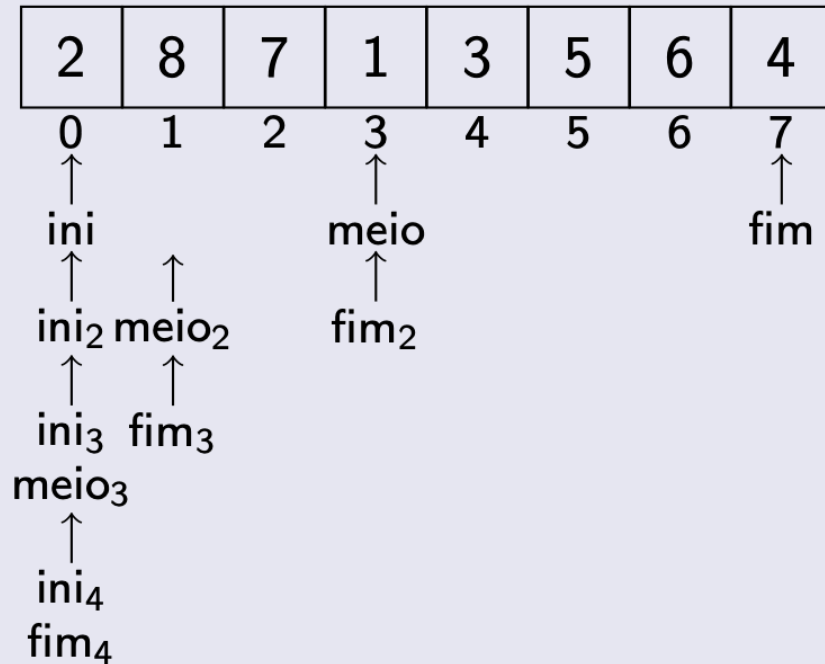
        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```



Merge Sort - Exemplo

```
void mergeSort(int numeros[],  
               int ini, int fim) {  
    int meio;  
    if(ini < fim) {  
        meio = (ini + fim)/2;  
  
        mergeSort(numeros, ini,  
                  meio);  
        mergeSort(numeros, meio+1,  
                  fim);  
  
        merge(numeros, ini, meio,  
              fim);  
    }  
}
```

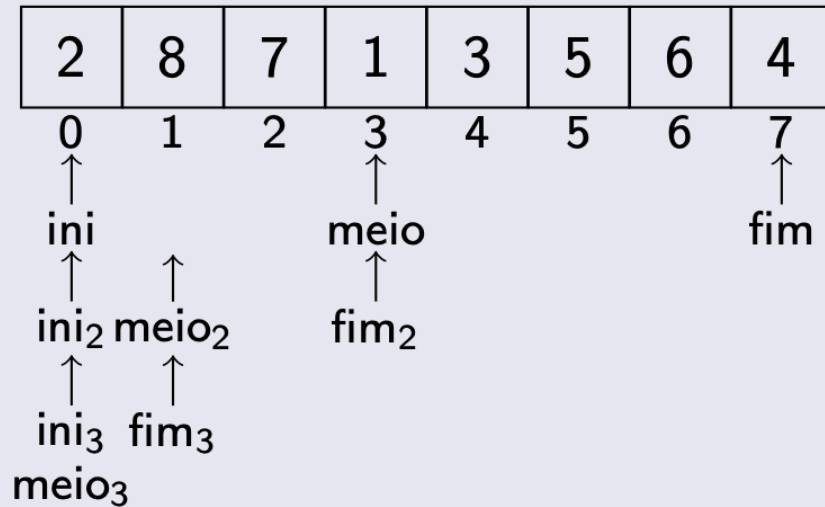


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

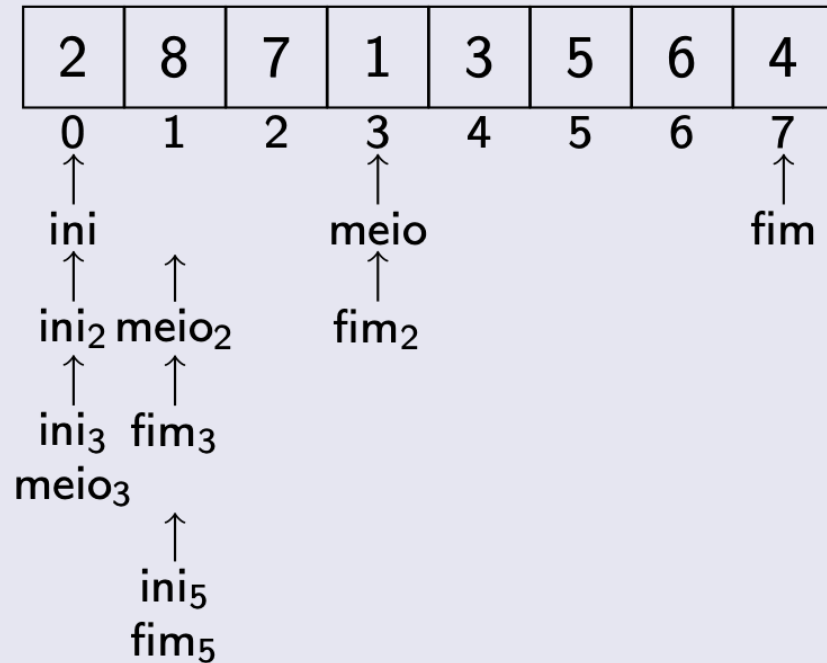
        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```



Merge Sort - Exemplo

```
void mergeSort(int numeros[],  
               int ini, int fim) {  
    int meio;  
    if(ini < fim) {  
        meio = (ini + fim)/2;  
  
        mergeSort(numeros, ini,  
                  meio);  
        mergeSort(numeros, meio+1,  
                  fim);  
  
        merge(numeros, ini, meio,  
              fim);  
    }  
}
```

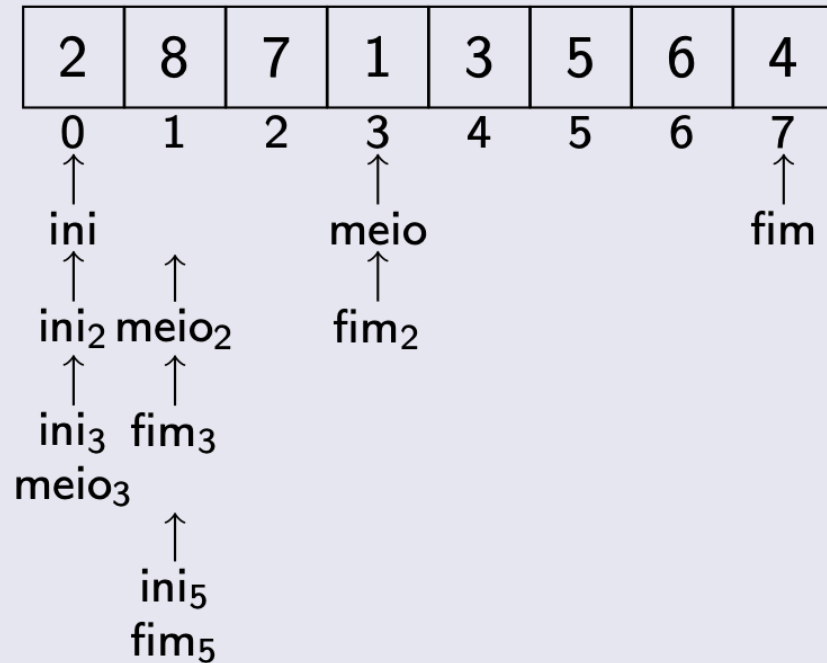


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

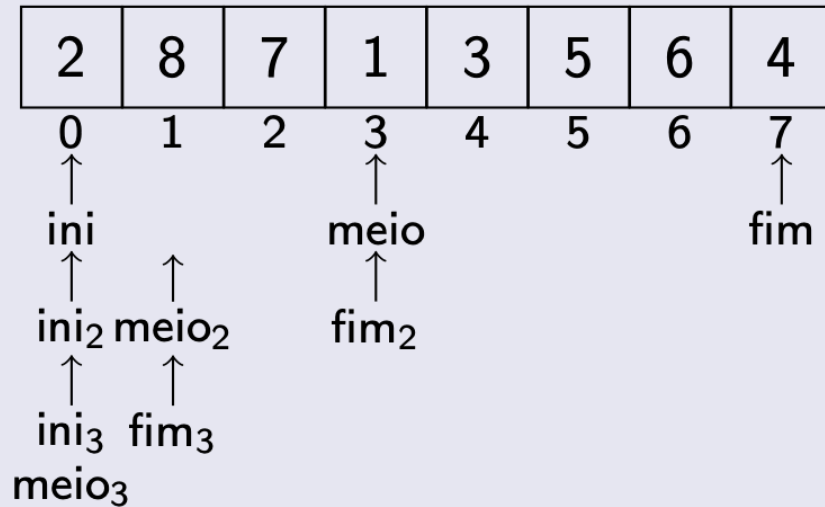
        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```



Merge Sort - Exemplo

```
void mergeSort(int numeros[],  
               int ini, int fim) {  
    int meio;  
    if(ini < fim) {  
        meio = (ini + fim)/2;  
  
        mergeSort(numeros, ini,  
                  meio);  
        mergeSort(numeros, meio+1,  
                  fim);  
  
        merge(numeros, ini, meio,  
              fim);  
    }  
}
```

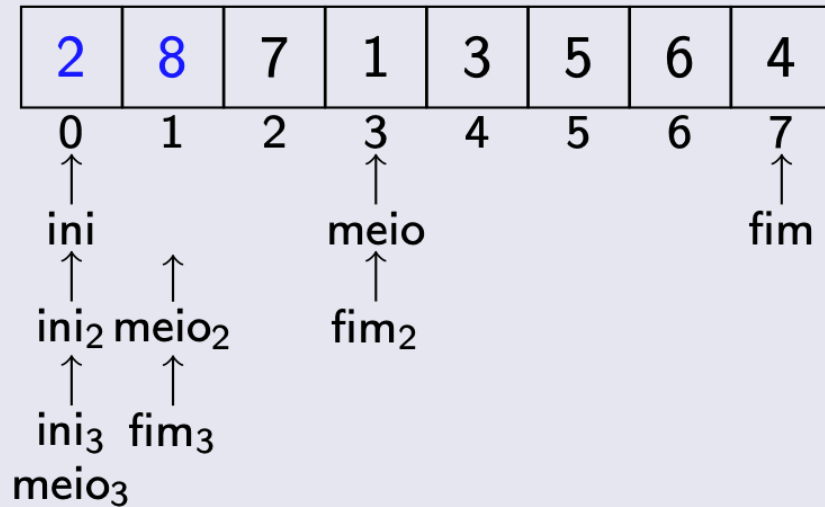


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

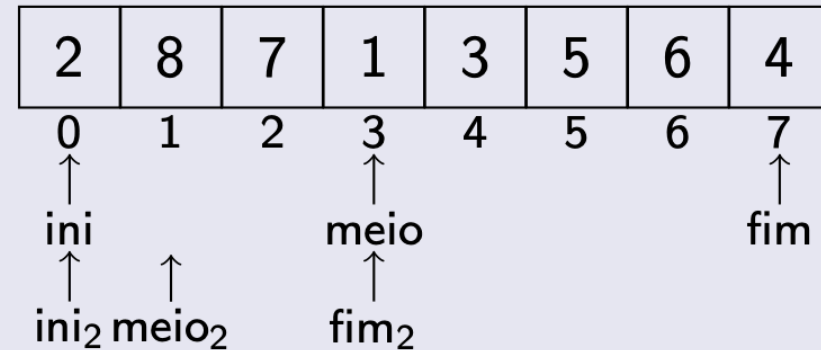


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

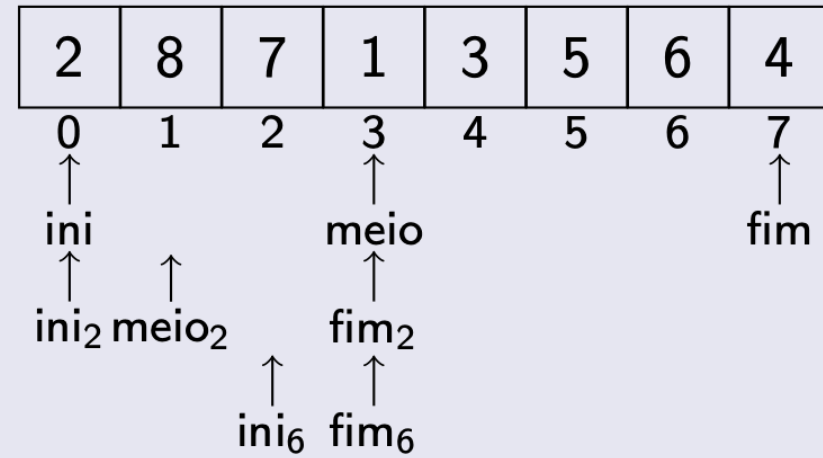


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

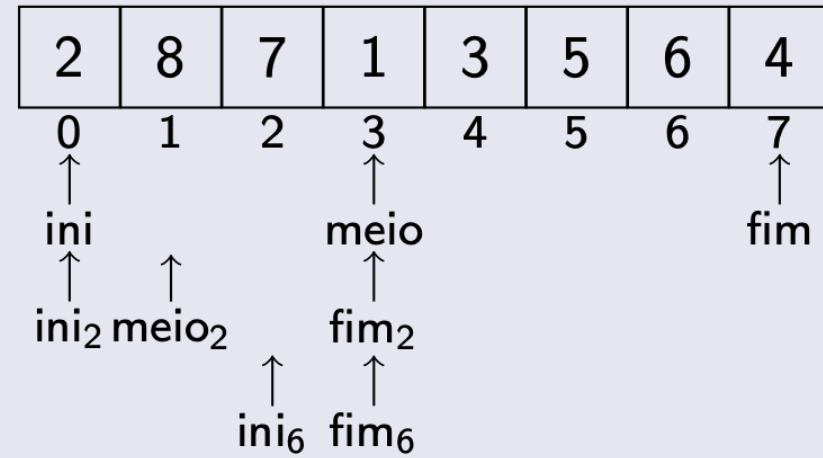


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

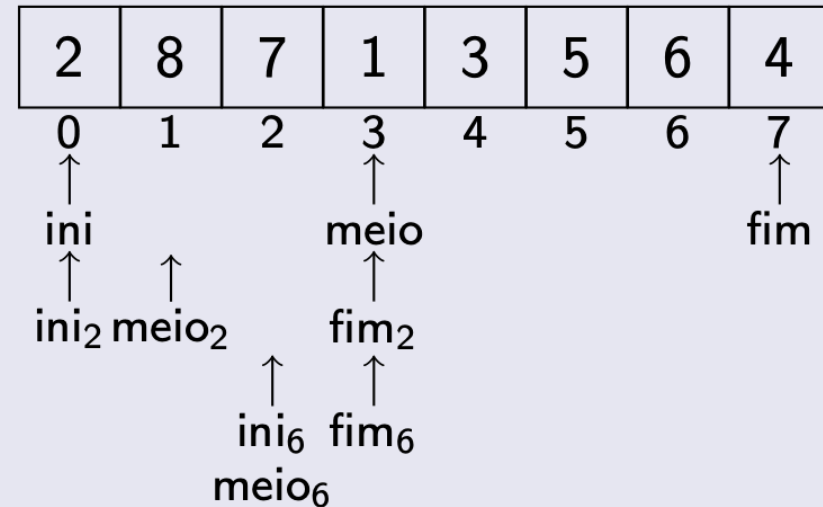


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

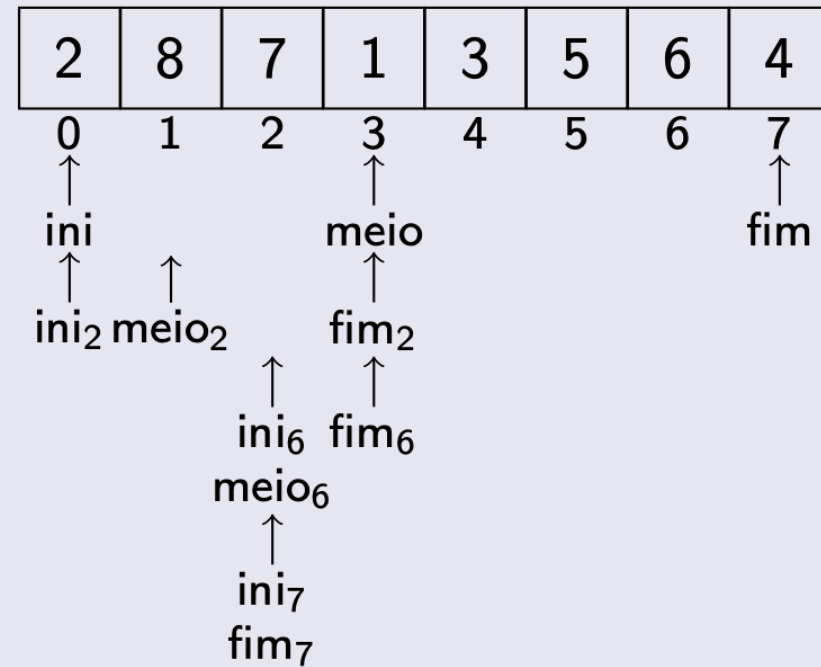


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

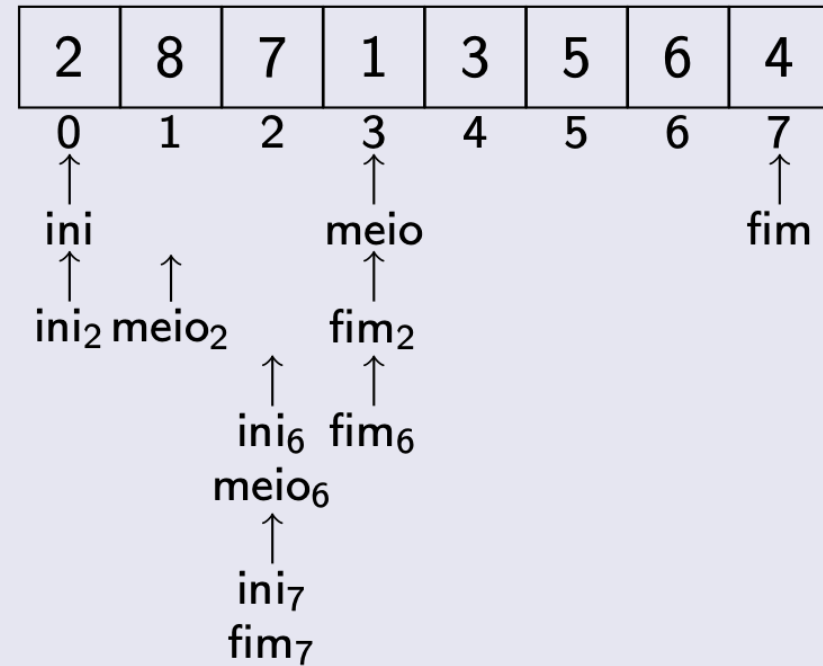


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

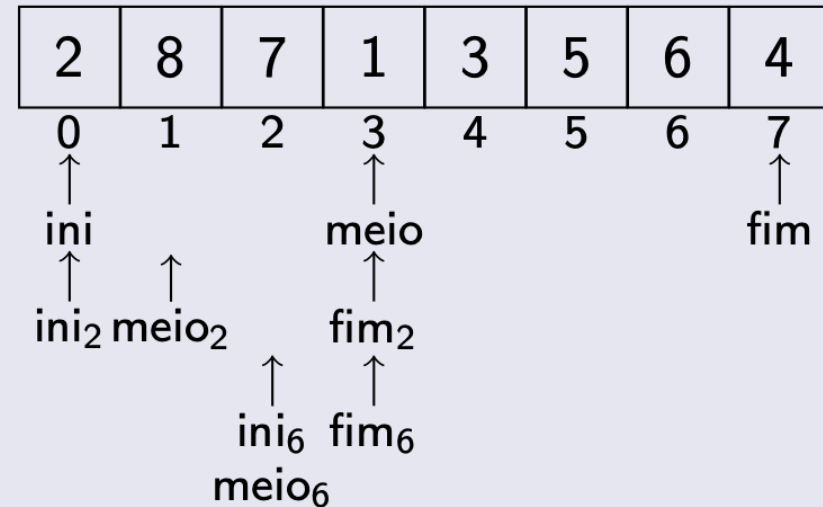


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

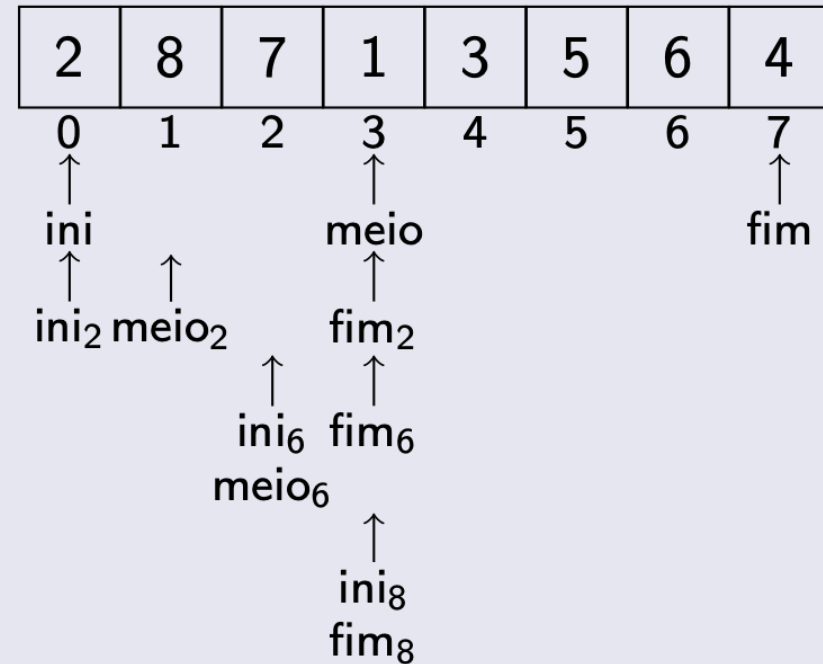


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

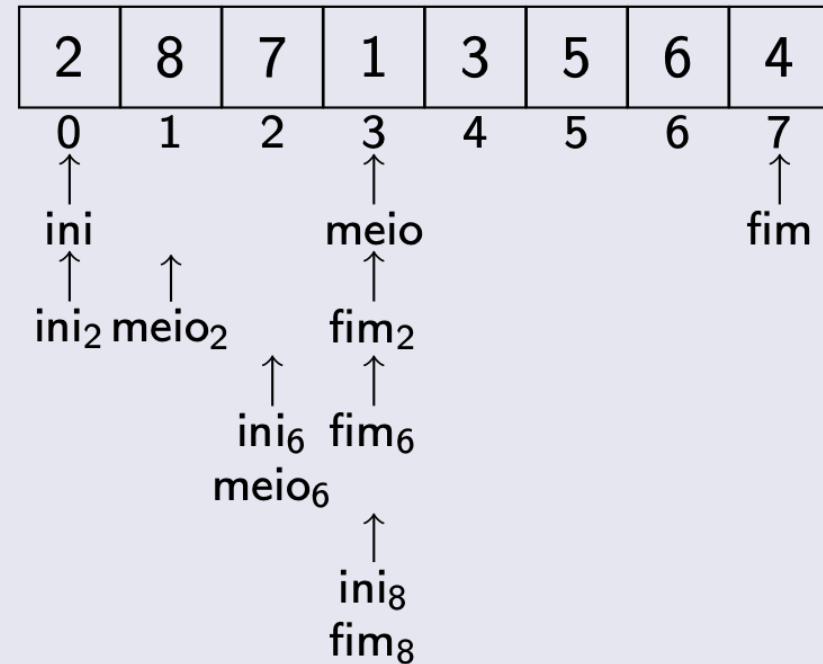


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

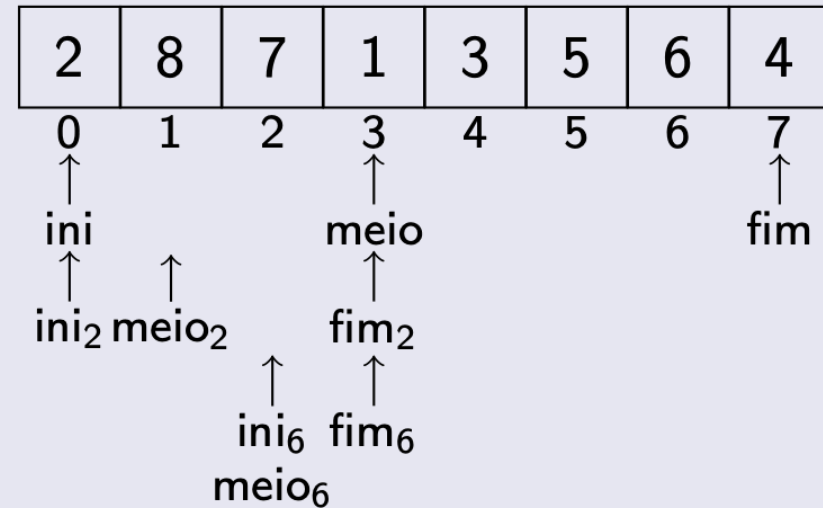


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

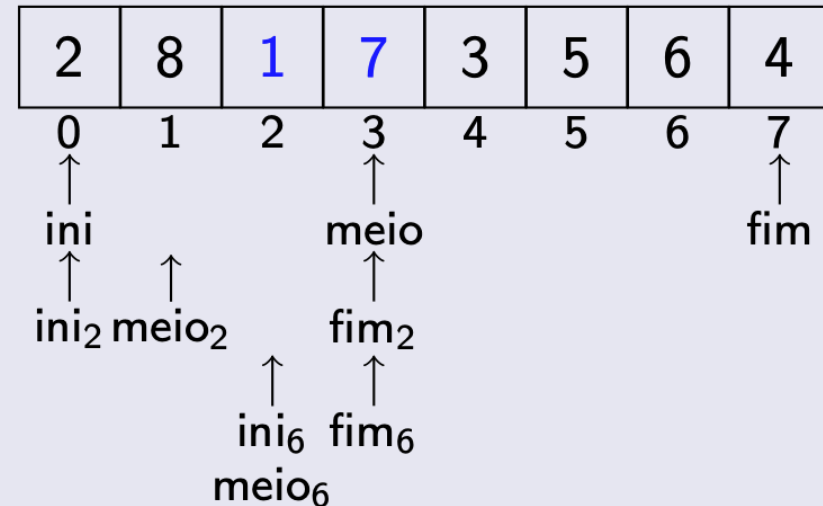


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

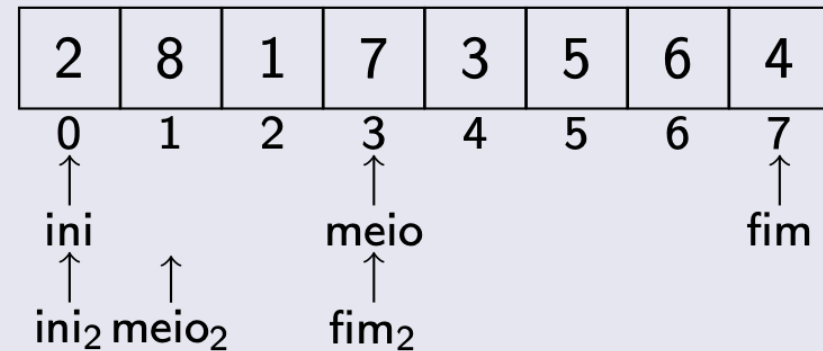


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

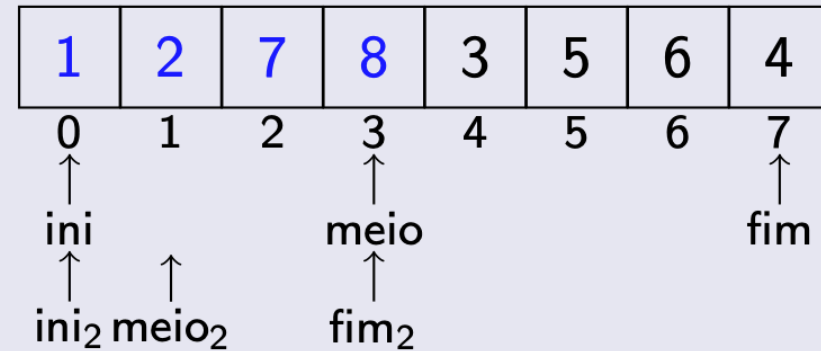


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```



Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

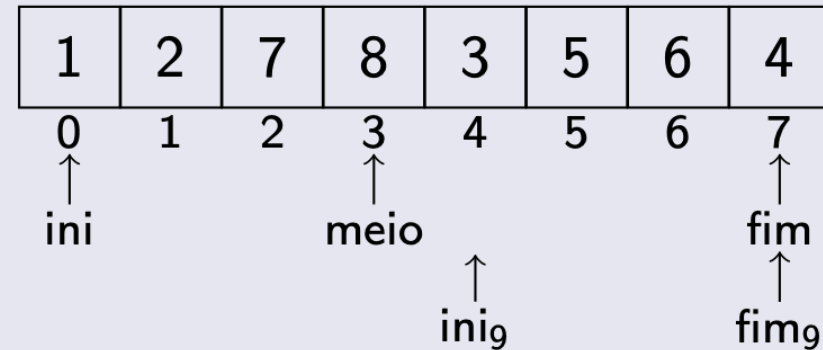
1	2	7	8	3	5	6	4
0	1	2	3	4	5	6	7
↑ ini			↑ meio				↑ fim

Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

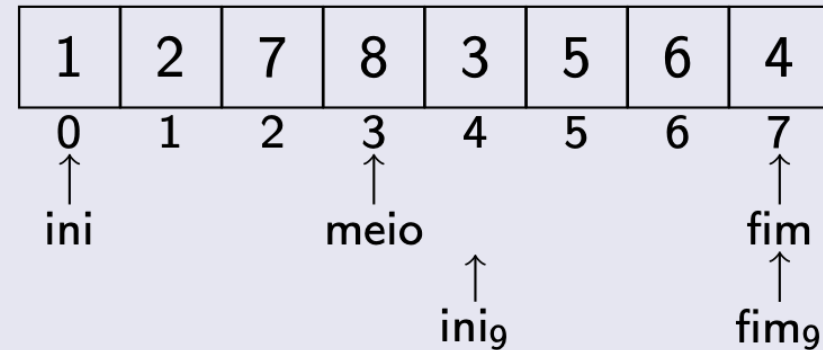


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

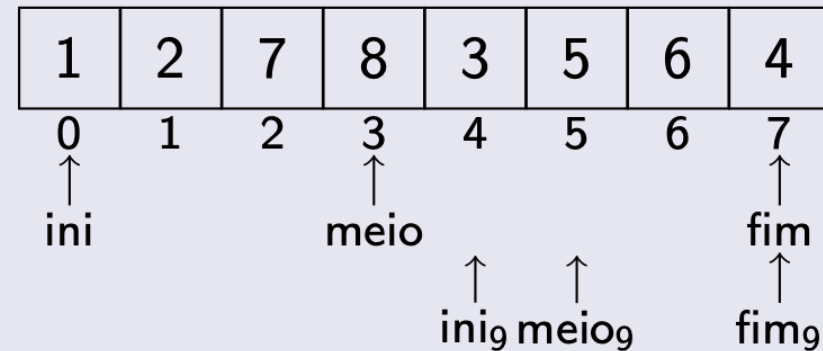


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

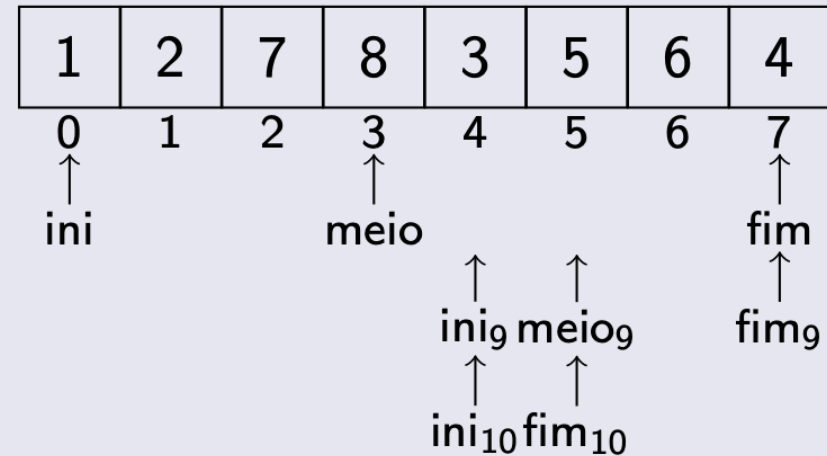


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

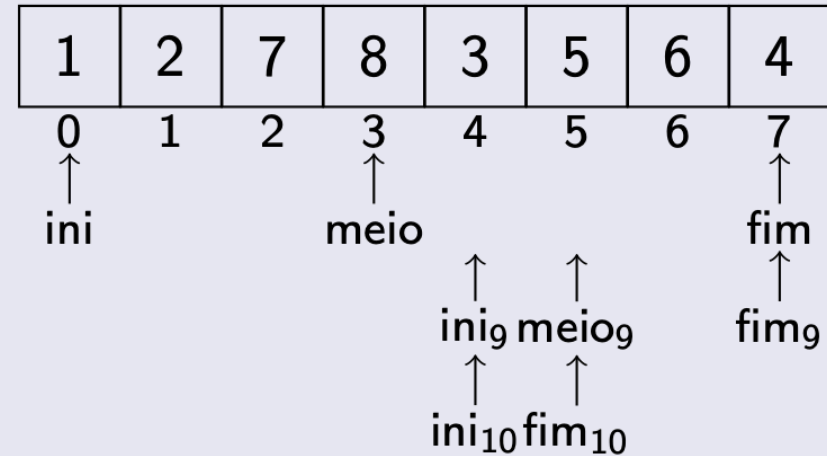


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

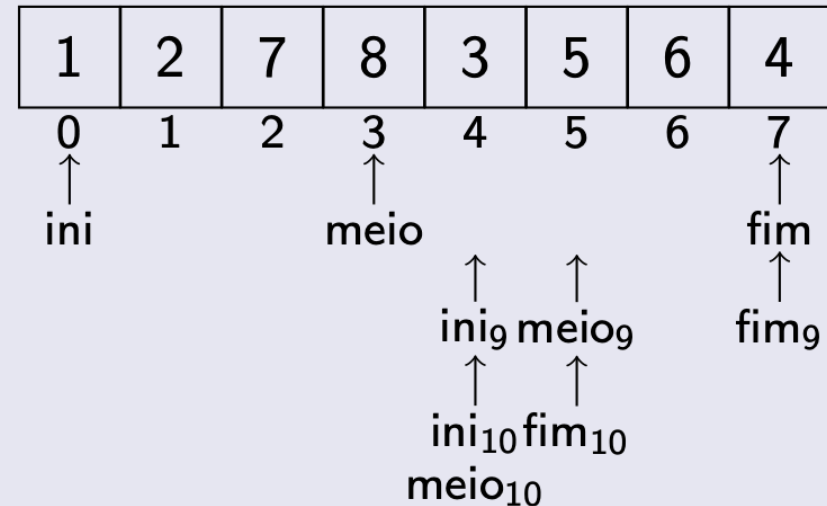


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

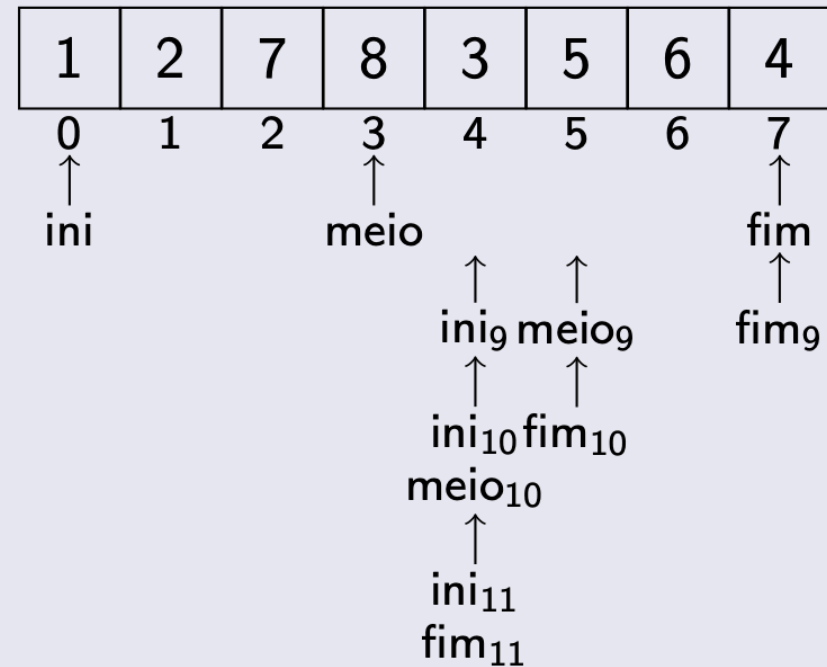


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

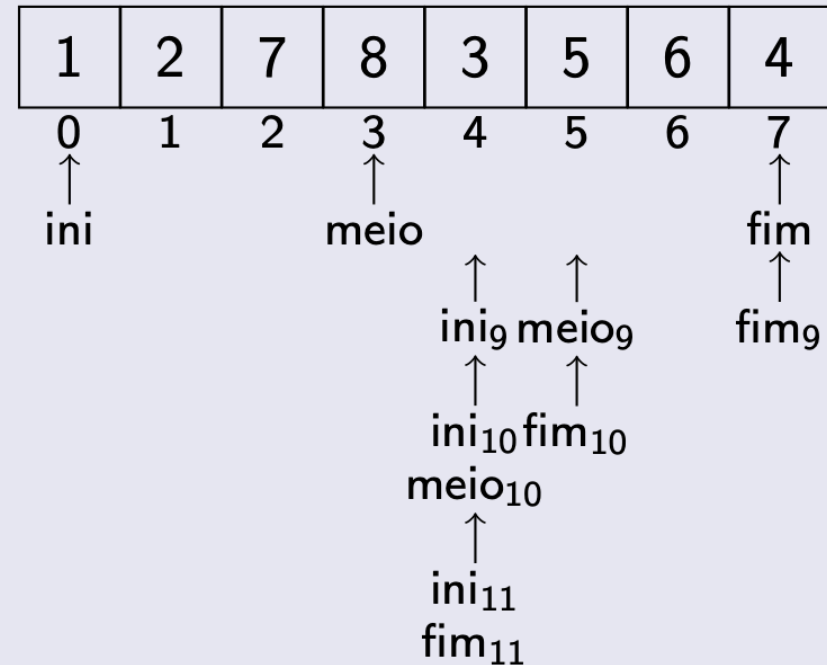


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

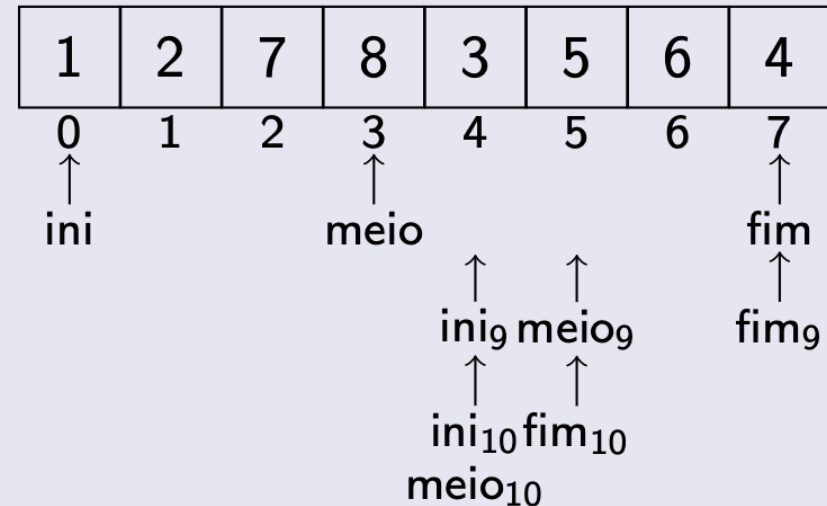


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

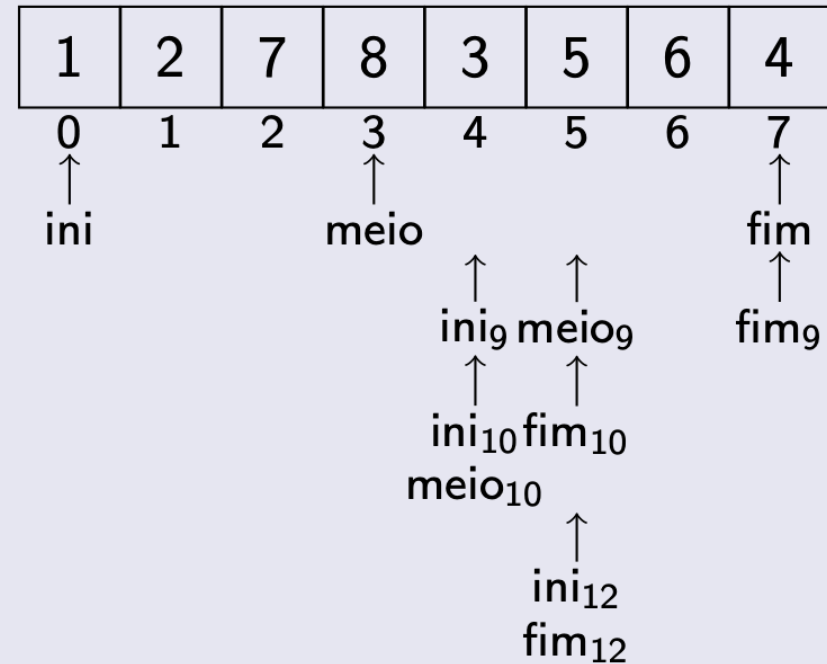


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

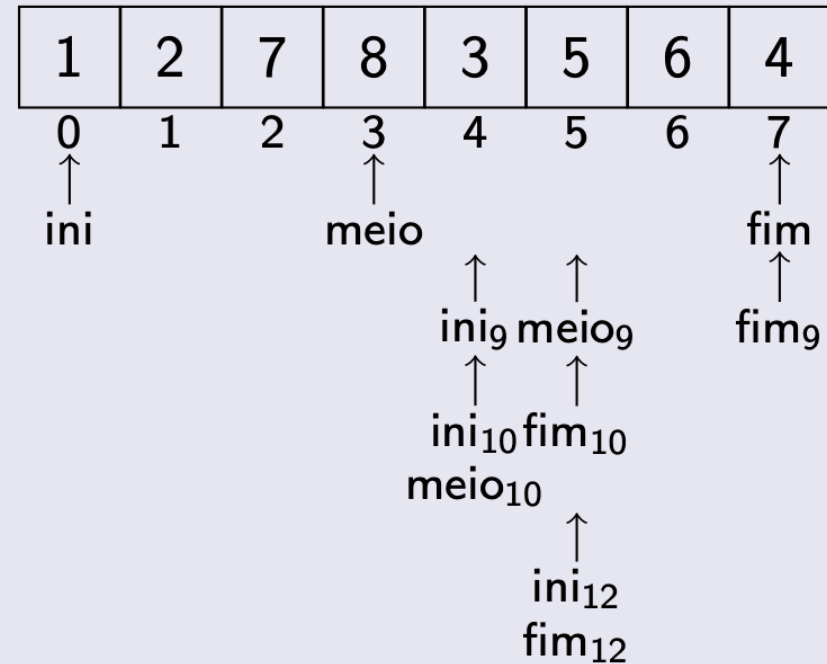


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

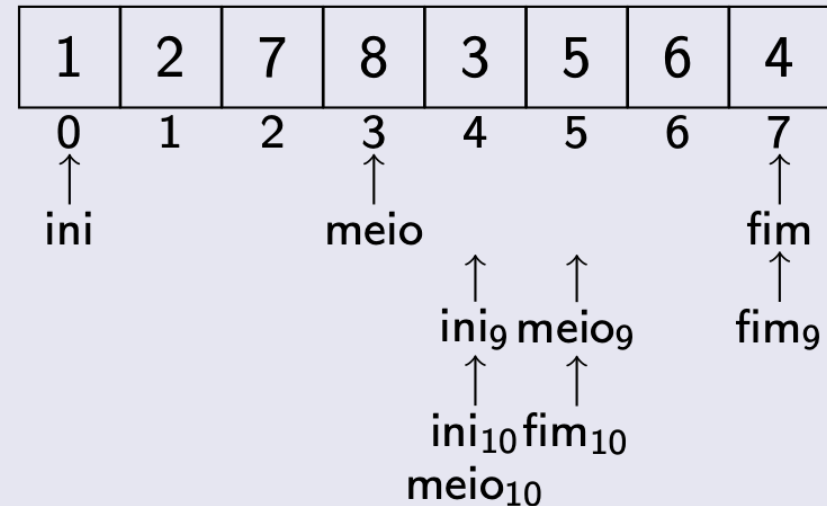


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```



Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

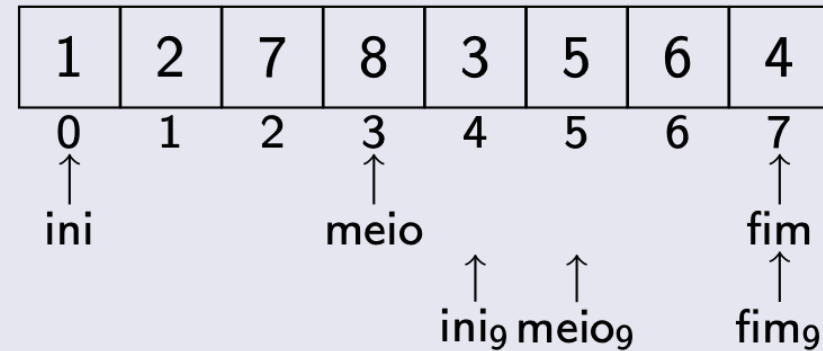


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

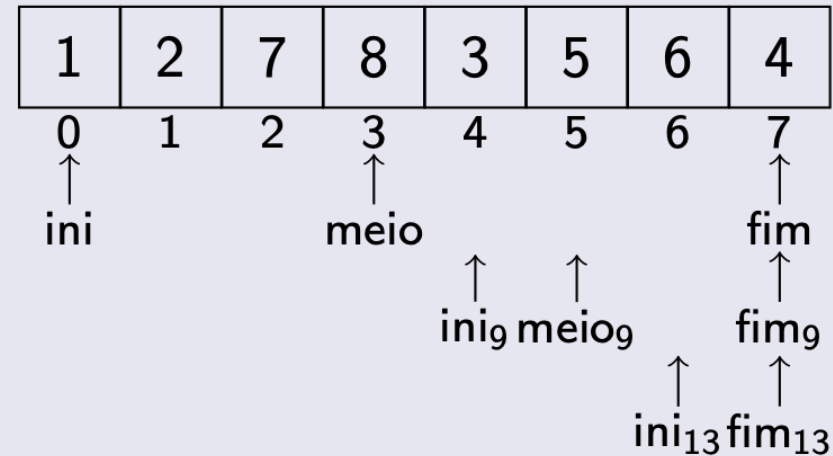


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

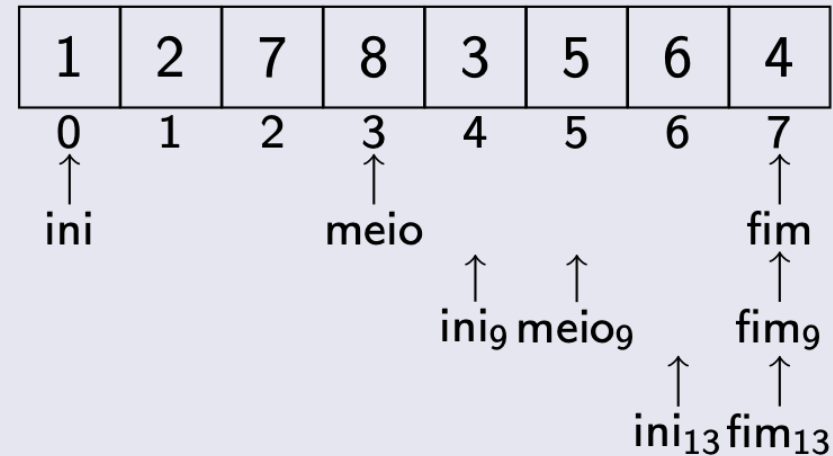


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

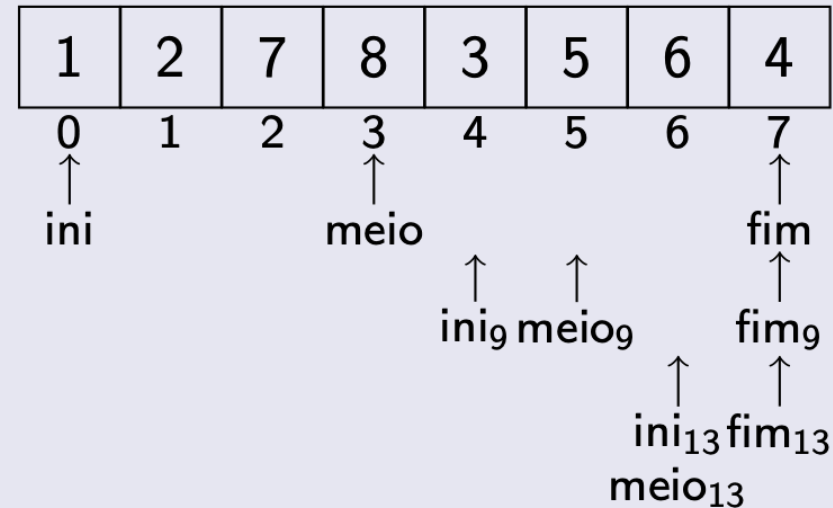


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```



Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```



Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

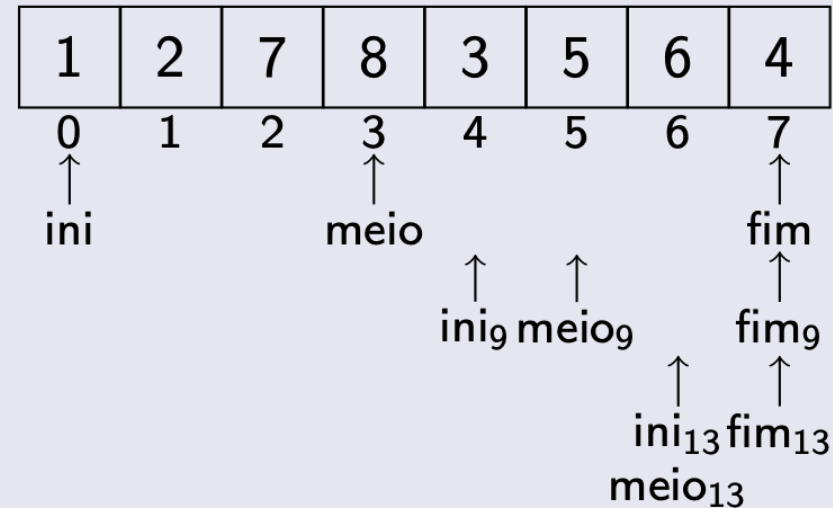


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

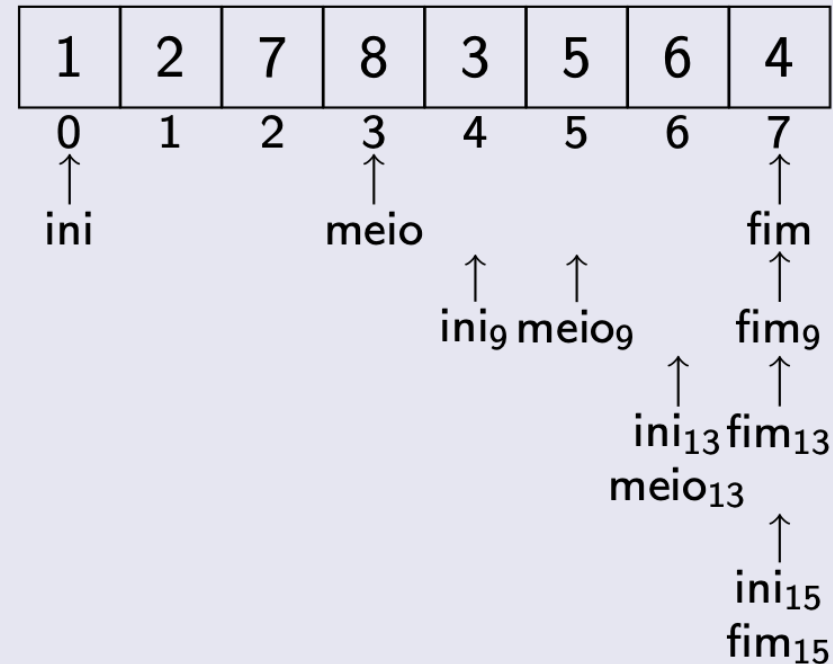


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

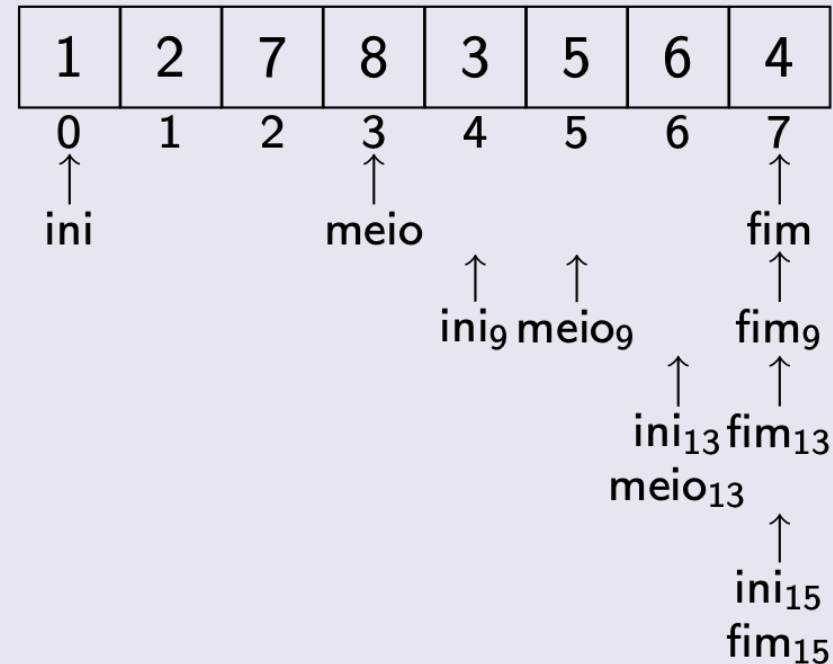


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

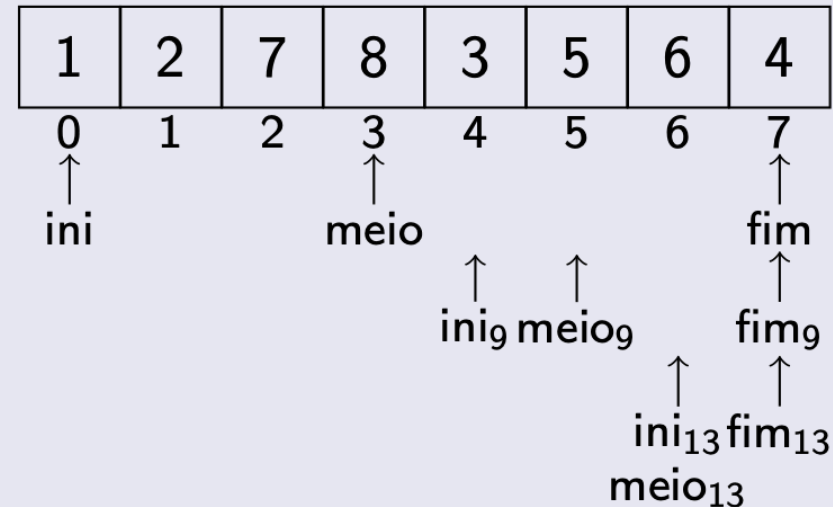


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

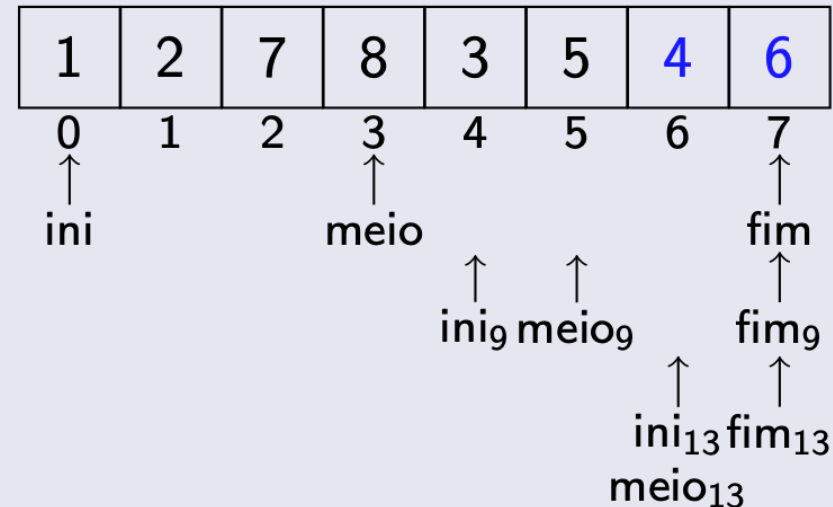


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

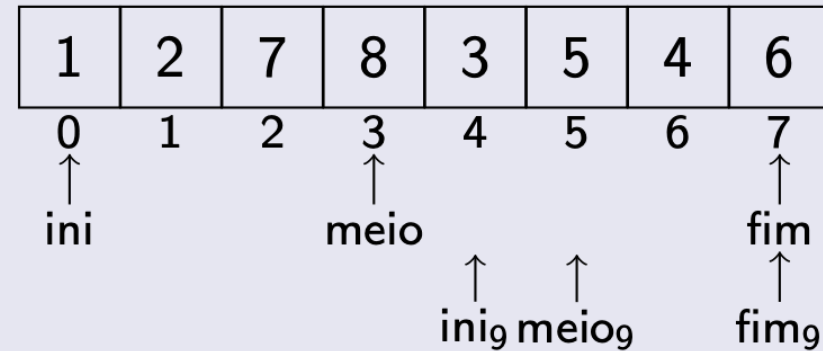


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

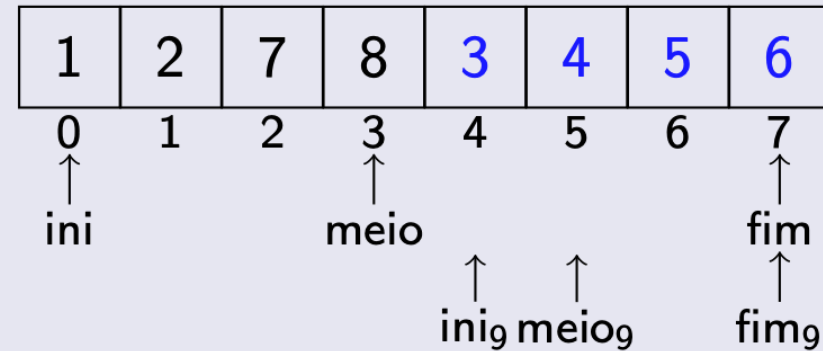


Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```



Merge Sort - Exemplo

```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

        merge(numeros, ini, meio,
              fim);
    }
}
```

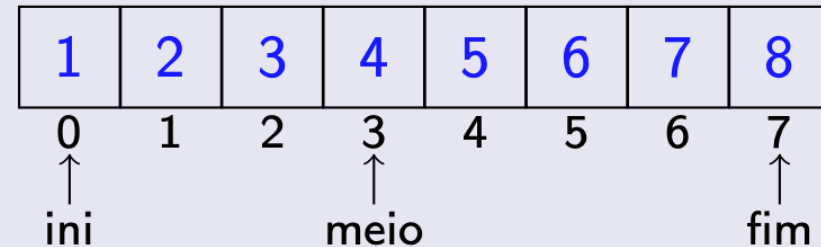
1	2	7	8	3	4	5	6
0	1	2	3	4	5	6	7
↑ ini			↑ meio				↑ fim

Merge Sort - Exemplo

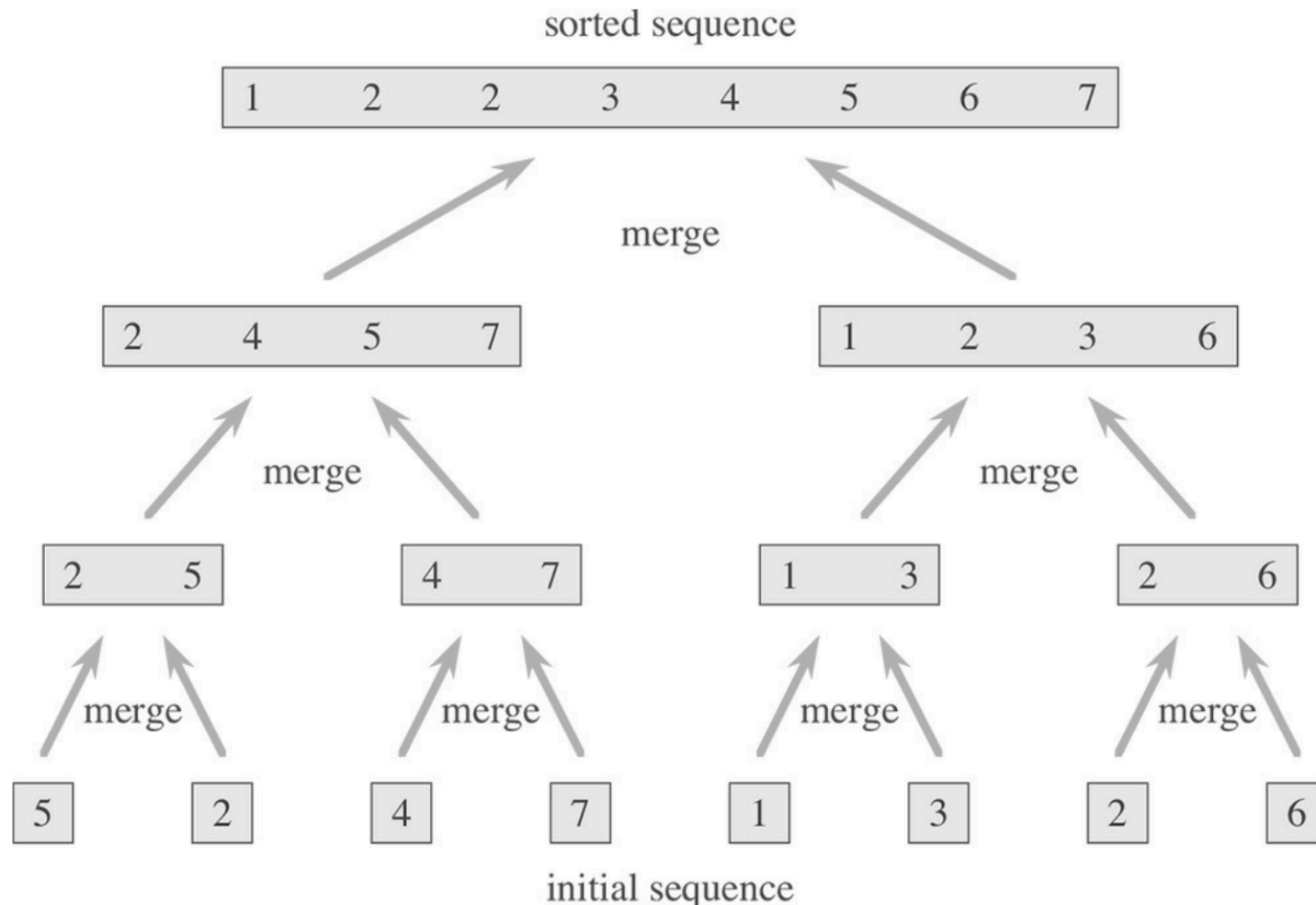
```
void mergeSort(int numeros[],
               int ini, int fim) {
    int meio;
    if(ini < fim) {
        meio = (ini + fim)/2;

        mergeSort(numeros, ini,
                  meio);
        mergeSort(numeros, meio+1,
                  fim);

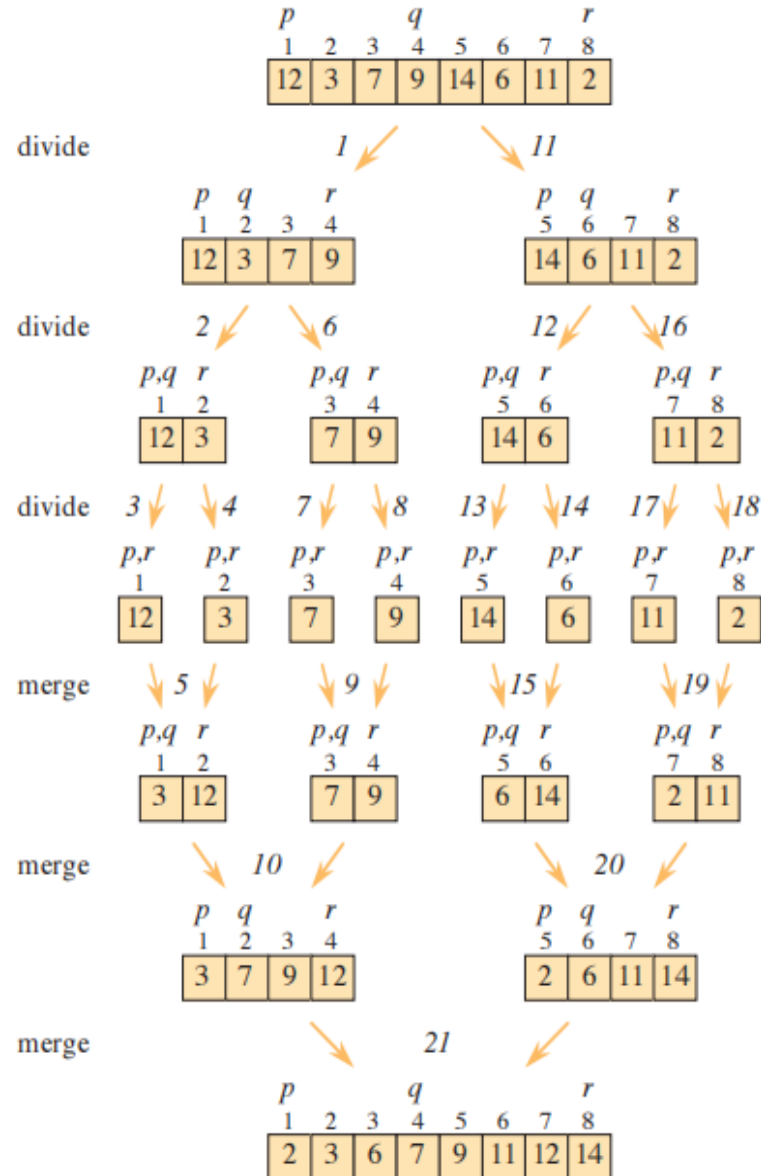
        merge(numeros, ini, meio,
              fim);
    }
}
```



Merge Sort - Complexidade



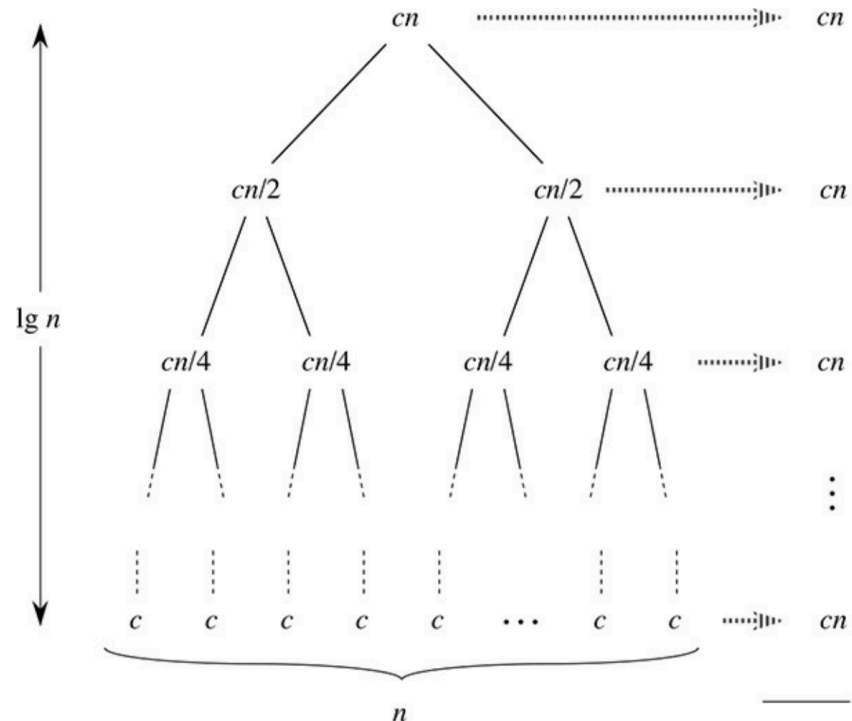
Merge Sort - Complexidade



Merge Sort - Complexidade

	Melhor	Médio	Pior
Merge Sort	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n \log_2 n)$

$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 1, \\ 2T(n/2) + \Theta(n) & \text{se } n > 1. \end{cases}$$



Exemplos didáticos

- Merge Sort:
 - <https://www.youtube.com/watch?v=dENca26N6V4>
(determinação do meio levemente diferente da implementação apresentada)
- Comparativo entre algoritmos de ordenação:
 - *Visualization and Comparison of Sorting Algorithms:*
<https://www.youtube.com/watch?v=ZZuD6iUe3Pc>
 - *15 Sorting Algorithms in 6 Minutes:*
<https://www.youtube.com/watch?v=kPRA0W1kECg>

Referências

- Cormen, Thomas H., Leiserson, Charles E., Rivest, Ronald L. Algoritmos: Teoria e Prática. Tradução da 3a edição americana. Elsevier Editora Ltda. 2012.